

Flutter - Würfelsimulator App

In diesem Kapitel erstellen wir eine einfache Flutter-App, die einen **Würfelsimulator** darstellt. Der Benutzer kann durch einen Klick auf den Würfel eine zufällige Zahl zwischen 1 und 6 erzeugen. Wir starten dabei mit einem leeren `main.dart`-File und fügen nach und nach die notwendigen sgn. **Widgets** hinzu.

Neue Flutter-App erstellen

Erstelle eine neue Flutter-App mit dem Namen `wuerfelsimulator`:

```
flutter create wuerfelsimulator
```

Navigiere in das Verzeichnis der neuen App und öffne VS Code:

```
cd wuerfelsimulator  
code .
```

Ersetze den Inhalt der `lib/main.dart`-Datei mit folgendem Code:

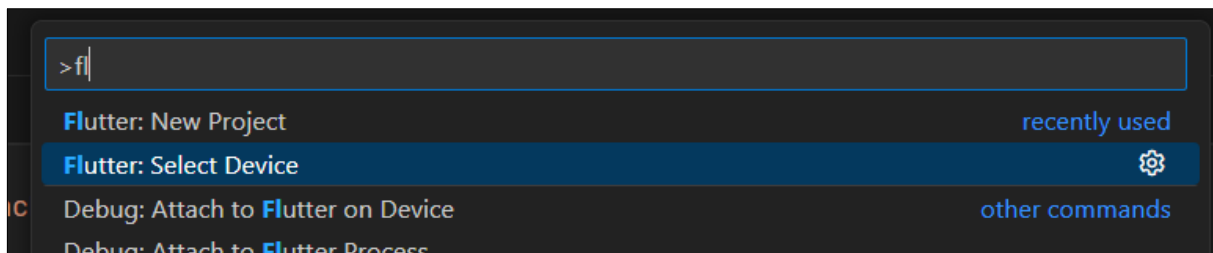
```
import 'package:flutter/material.dart';  
  
void main() {  
  runApp(MaterialApp(  
    home: Text('Hello beautiful world')));  
}
```

In Flutter wird die Benutzeroberfläche durch sogenannte **Widgets** erstellt. Ein Widget ist eine Klasse, die eine bestimmte visuelle Komponente darstellt. Widgets können andere Widgets enthalten, um komplexere Benutzeroberflächen zu erstellen. Flutter bietet eine Vielzahl von vordefinierten Widgets, die wir verwenden können, um unsere Benutzeroberfläche zu erstellen. Wir können aber auch eigene Widgets erstellen.

Das **MaterialApp**-Widget ist das oberste Widget in einer Flutter-App. Es definiert das Aussehen und das Verhalten der App. Praktisch jede Flutter-App beginnt mit einem **MaterialApp**-Widget.

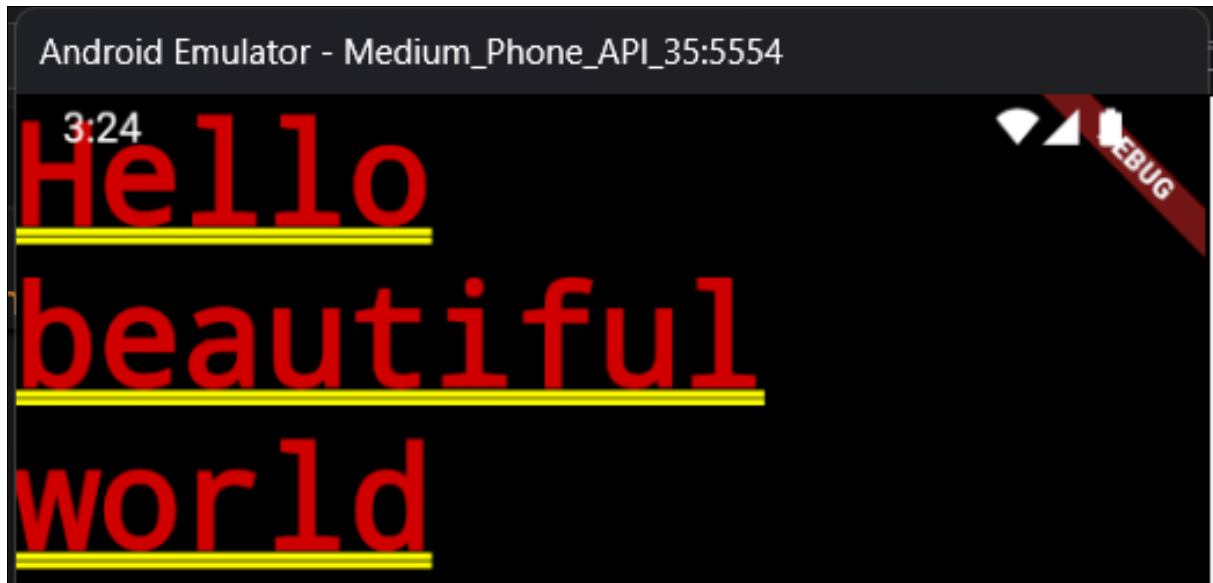
Emulator starten

Wähle nun den Emulator aus ([F1] oder [Strg]+[Shift]+[P] und dann "Flutter: Select Device").



Starte nun die App mit [F5].

Das Ergebnis ist ... nun ja ... nicht so schön, aber das werden wir gleich ändern.



Das Scaffold-Widget

Das **Scaffold**-Widget definiert die grundlegende Struktur der Benutzeroberfläche einer App. Es enthält normalerweise eine App-Leiste, einen Hauptbereich und einen unteren Navigationsbereich.

```
import 'package:flutter/material.dart';

void main() {
  runApp(MaterialApp(
    home: Scaffold(
      appBar: AppBar(
        title: const Text('Würfelsimulator'),
      ),
      body: const Center(
        child: const Text('Hello beautiful world'),
      ),
    ),
  ));
}
```

HINWEIS: In Flutter können wir das `const`-Schlüsselwort verwenden, um Widgets zu erstellen, die sich nicht ändern. Dies kann die Leistung verbessern, da Flutter die Widgets wiederverwenden kann, anstatt sie jedes Mal neu zu erstellen.

Das **MaterialApp**-Widget enthält normalerweise ein **Scaffold**-Widget, das die eigentliche Benutzeroberfläche der App definiert.

Zur Erinnerung: Eine neue Instanz einer Klasse wird in Dart ohne das Schlüsselwort `new` erstellt. Die Parameter können über deren Namen übergeben werden.

Das Center-Widget

Das **Center**-Widget zentriert sein Kind in der Mitte des Bildschirms. In diesem Fall wird ein **Text**-Widget mit dem Text "Würfel" zentriert.

Das AppBar-Widget

Das **AppBar**-Widget definiert die App-Leiste, die oben auf dem Bildschirm angezeigt wird. Das **AppBar**-Widget enthält normalerweise ein **Text** oder ein **Icon**, das den Titel der App darstellt.

Das Text-Widget

Das **Text**-Widget zeigt einen Text auf dem Bildschirm an. In diesem Fall wird der Text "Würfel" angezeigt.

Der "Flutter Widget Catalog"

Der [Flutter Widget Catalog](#) ist eine praktische Ressource, um die verschiedenen Widgets zu erkunden, die Flutter bietet. Der Katalog enthält eine Liste aller Widgets, die in Flutter verfügbar sind, und zeigt Beispiele für deren Verwendung.

Zum Beispiel findet sich die Dokumentation für das Text-Widget [hier](#).

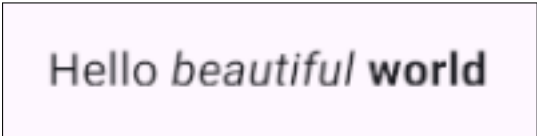
Um zum Beispiel den Text *"Hello beautiful world"* mit unterschiedlichen *Styles* anzuzeigen, können wir den `style`-Parameter des Text-Widgets verwenden:

```
Text(  
  'Hello beautiful world',  
  style: TextStyle(  
    fontSize: 24,
```

```
        color: Colors.blue,  
        fontWeight: FontWeight.bold,  
      ),  
    ),
```

Man kann sogar noch weiter gehen wie hier gezeigt und einen **alternativen Konstruktor** verwenden, um den Text zu formatieren:

```
Text.rich(  
  TextSpan(  
    text: 'Hello', // default text style  
    children: <TextSpan>[  
      TextSpan(text: ' beautiful ', style: TextStyle(fontStyle:  
→ FontStyle.italic)),  
      TextSpan(text: 'world', style: TextStyle(fontWeight:  
→ FontWeight.bold)),  
    ],  
  ),  
)
```



Hintergrundfarbe ändern

Um die Hintergrundfarbe der App zu ändern, können wir das **Scaffold**-Widget verwenden und die **backgroundColor**-Eigenschaft auf eine beliebige Farbe setzen:

```
import 'package:flutter/material.dart';
```

```
void main() {
```

```
runApp(MaterialApp(  
  home: Scaffold(  
    appBar: AppBar(  
      title: Text('Würfelsimulator'),  
    ),  
    body: Center(  
      child: Text('Hello beautiful world'),  
    ),  
    backgroundColor: Colors.blueGrey,  
  ),  
));  
}
```

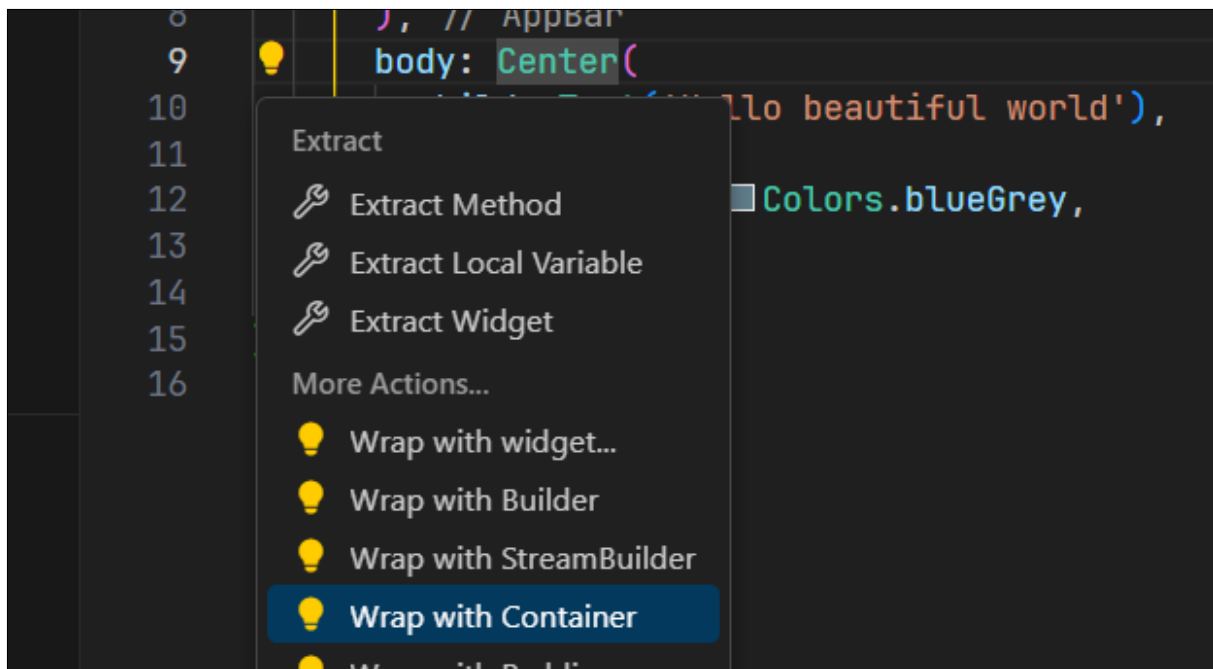
Neben den vordefinierten Farben in der **Colors**-Klasse können wir auch benutzerdefinierte Farben erstellen:

```
Color myColor = Color(0xFF123456);
```

Etwas lesbarer wird es, wenn wir z.B. die Methode `Color.fromARGB()` verwenden:

```
Color myColor = Color.fromARGB(255, 18, 52, 86);
```

Wenn wir nun aber einen **Farbverlauf** verwenden wollen, können wir z.B. das `Center`-Widget in ein `Container`-Widget umschließen und die **decoration**-Eigenschaft des Containers verwenden:



Stylen wir nun auch noch den Text etwas um:

```
import 'package:flutter/material.dart';
```

```
void main() {  
  runApp(MaterialApp(  
    home: Scaffold(  
      appBar: AppBar(  
        title: Text('Würfelsimulator'),  
      ),  
      body: Container(  
        decoration: BoxDecoration(  
          gradient: LinearGradient(  
            begin: Alignment.topCenter,  
            end: Alignment.bottomCenter,  
            colors: [Colors.blue, Colors.blueGrey],  
          ),  
        ),  
      ),  
    ),  
  ),  
)
```

```
        child: Center(  
          child: Text(  
            'Hello beautiful world',  
            style: TextStyle(  
              fontSize: 30,  
              color: Colors.white,  
              fontWeight: FontWeight.bold,  
            ),  
          ),  
        ),  
      ),  
    ),  
  ),  
));  
}
```

Das Container-Widget wird häufig verwendet, um die Darstellung von Widgets zu ändern. Es kann z.B. verwendet werden, um einen Hintergrund oder einen Rahmen hinzuzufügen. Folgende neue Widgets haben wir im oberen Beispiel verwendet:

- **BoxDecoration:** Definiert die Dekoration eines Containers.
- **LinearGradient:** Definiert einen linearen Farbverlauf.
- **Alignment:** Definiert die Ausrichtung eines Widgets.

Wie man sieht, können wir mit Flutter sehr einfach komplexe Benutzeroberflächen erstellen. Die einfache Syntax zum Erstellen von Objekten in Kombination mit sinnvollem Einrücken des Codes machen die Erstellung von Benutzeroberflächen in Flutter meiner Meinung nach sehr angenehm.

Der Nachteil ist jedoch, dass unsere App langsam aber doch sehr unübersichtlich wird. Daher werden wir in den nächsten Schritten die verschiedenen Teile der App in separate Widgets extrahieren.

Benutzerdefinierte Widgets

In Flutter können wir benutzerdefinierte Widgets erstellen, um unseren Code zu organisieren und zu strukturieren. Ein benutzerdefiniertes Widget ist eine Klasse, die von der **StatelessWidget**- oder **StatefulWidget**-Klasse erbt und die **build**-Methode überschreibt.

Extrahieren wir nun den body unseres Scaffold-Widgets in ein benutzerdefiniertes Widget namens **DicePage**. Erstelle eine neue Datei namens `dice_page.dart` im `lib`-Verzeichnis und füge folgenden Code hinzu:

```
import 'package:flutter/material.dart';

class DicePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Container(
      decoration: BoxDecoration(
        gradient: LinearGradient(
          begin: Alignment.topCenter,
          end: Alignment.bottomCenter,
          colors: [Colors.blue, Colors.blueGrey],
        ),
      ),
      child: Center(
        child: Text(
          'Hello beautiful world',
          style: TextStyle(
            fontSize: 30,
            color: Colors.white,
            fontWeight: FontWeight.bold,
          ),
        ),
      ),
    ),
  ),
}
```

```
    ),  
  );  
}  
}
```

Importiere das **DicePage**-Widget in der `main.dart`-Datei und verwende es als body des **Scaffold**-Widgets:

```
import 'package:flutter/material.dart';  
import 'dice_page.dart';  
  
void main() {  
  runApp(MaterialApp(  
    home: Scaffold(  
      appBar: AppBar(  
        title: Text('Würfelsimulator'),  
      ),  
      body: DicePage(),  
    ),  
  ));  
}
```

Nun ist unsere App schon deutlich übersichtlicher und einfacher zu lesen. Jedoch ist der Text *"Hello beautiful world"* nach wie vor hartcodiert. Wir wollen aber einen beliebigen Text übergeben können. Dazu fügen wir einen Konstruktor zum **DicePage**-Widget hinzu:

```
import 'package:flutter/material.dart';  
  
class DicePage extends StatelessWidget {  
  final String text;  
  
  const DicePage({required this.text, super.key});  
}
```

@override

```
Widget build(BuildContext context) {  
  return Container(  
    decoration: BoxDecoration(  
      gradient: LinearGradient(  
        begin: Alignment.topCenter,  
        end: Alignment.bottomCenter,  
        colors: [Colors.blue, Colors.blueGrey],  
      ),  
    ),  
    child: Center(  
      child: Text(  
        text,  
        style: TextStyle(  
          fontSize: 30,  
          color: Colors.white,  
          fontWeight: FontWeight.bold,  
        ),  
      ),  
    ),  
  );  
}
```

Hinweis: Im Konstruktor werden sgn. **benannte Parameter** verwendet. Diese sind optional und können in beliebiger Reihenfolge übergeben werden. Im Beispiel oben erben wir von der Klasse **StatelessWidget** und verwenden den Konstruktor **super.key**. Eine alternative Schreibweise wäre gewesen: `DicePage({required this.text, Key? key}) : super(key: key);`, welche aber, wie man sieht, deutlich schlechter les-

bar ist.

Nun können wir den Text über den Konstruktor des **DicePage**-Widgets übergeben:

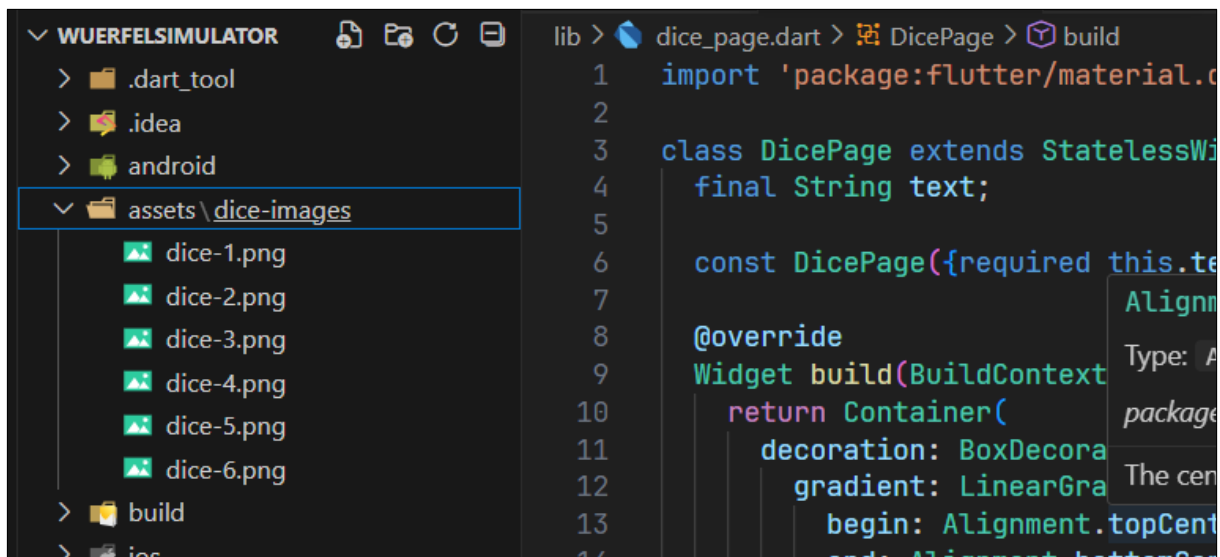
```
import 'package:flutter/material.dart';
import 'dice_page.dart';

void main() {
  runApp(MaterialApp(
    home: Scaffold(
      appBar: AppBar(
        title: Text('Würfelsimulator'),
      ),
      body: DicePage(text: 'Hello beautiful world'),
    ),
  ));
}
```

Images in Flutter

Unsere App soll ja bei jedem Klick auf einen Button einen zufälligen Würfel anzeigen. Dazu benötigen wir ein Bild jedes Würfels. Wir können Bilder in Flutter mit dem **Image**-Widget anzeigen. Das Image-Widget kann Bilder aus dem Netzwerk oder aus dem lokalen Speicher anzeigen.

Lade die Datei "**dice-images.zip**" herunter. Erstelle im Root-Verzeichnis deiner Flutter-App einen Ordner namens "**assets**" und entpacke den Inhalt der Datei "*dice-images.zip*" in diesen Ordner.



Damit diese Images auch in der App verwendet werden können, öffne die Datei **pubspec.yaml** und suche nach dem Abschnitt **assets**. Entferne den Kommentar und füge folgende Zeilen ein:

```
flutter:  
  assets:  
    - assets/dice-images/dice-1.png  
    - assets/dice-images/dice-2.png  
    - assets/dice-images/dice-3.png  
    - assets/dice-images/dice-4.png  
    - assets/dice-images/dice-5.png  
    - assets/dice-images/dice-6.png
```

Nun können wir z.B. den Text durch ein Bild ersetzen. Wir übergeben dem DicePage-Widget also nicht mehr den anzuzeigenden Text, sondern den Pfad zum Bild:

```
import 'package:flutter/material.dart';  
  
class DicePage extends StatelessWidget {  
  final String imagePath;
```

```
const DicePage({required this.imagePath, super.key});

@override
Widget build(BuildContext context) {
  return Container(
    decoration: BoxDecoration(
      gradient: LinearGradient(
        begin: Alignment.topCenter,
        end: Alignment.bottomCenter,
        colors: [Colors.blue, Colors.blueGrey],
      ),
    ),
    child: Center(
      child: Image.asset(imagePath),
    ),
  );
}
```

Hinweis: Wir verwenden hier den alternativen Konstruktor **Image.asset**, um ein Bild aus dem lokalen Speicher anzuzeigen. Der Pfad zum Bild wird relativ zum `pubspec.yaml`-File angegeben.

Nun können wir das **DicePage**-Widget in der `main.dart`-Datei verwenden und den Pfad zum Bild übergeben:

```
import 'package:flutter/material.dart';
import 'dice_page.dart';

void main() {
  runApp(MaterialApp(
```

```
home: Scaffold(  
  appBar: AppBar(  
    title: Text('Würfelsimulator'),  
  ),  
  body: DicePage(imagePath: 'assets/dice-images/dice-1.png'),  
),  
));  
}
```

Nach einem Hot-Reload unserer App sollten wir nun das Bild eines Würfels sehen. Dieser ist noch etwas groß. Wir können die Größe des Bildes mit dem **width**-Parameter des **Image**-Widgets ändern:

```
import 'package:flutter/material.dart';  
  
class DicePage extends StatelessWidget {  
  final String imagePath;  
  
  const DicePage({required this.imagePath, super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      decoration: BoxDecoration(  
        gradient: LinearGradient(  
          begin: Alignment.topCenter,  
          end: Alignment.bottomCenter,  
          colors: [Colors.blue, Colors.blueGrey],  
        ),  
      ),  
      child: Center(  

```

```
        child: Image.asset(  
          imagePath,  
          width: 200,  
        ),  
      ),  
    );  
  }  
}
```

Button hinzufügen

Nun fügen wir einen Button hinzu, um die Würfel zu werfen. Wir möchten den Button unterhalb des Würfels anzeigen, benötigen also ein neues **Column**-Widget, um den Würfel und den Button anzuzeigen.

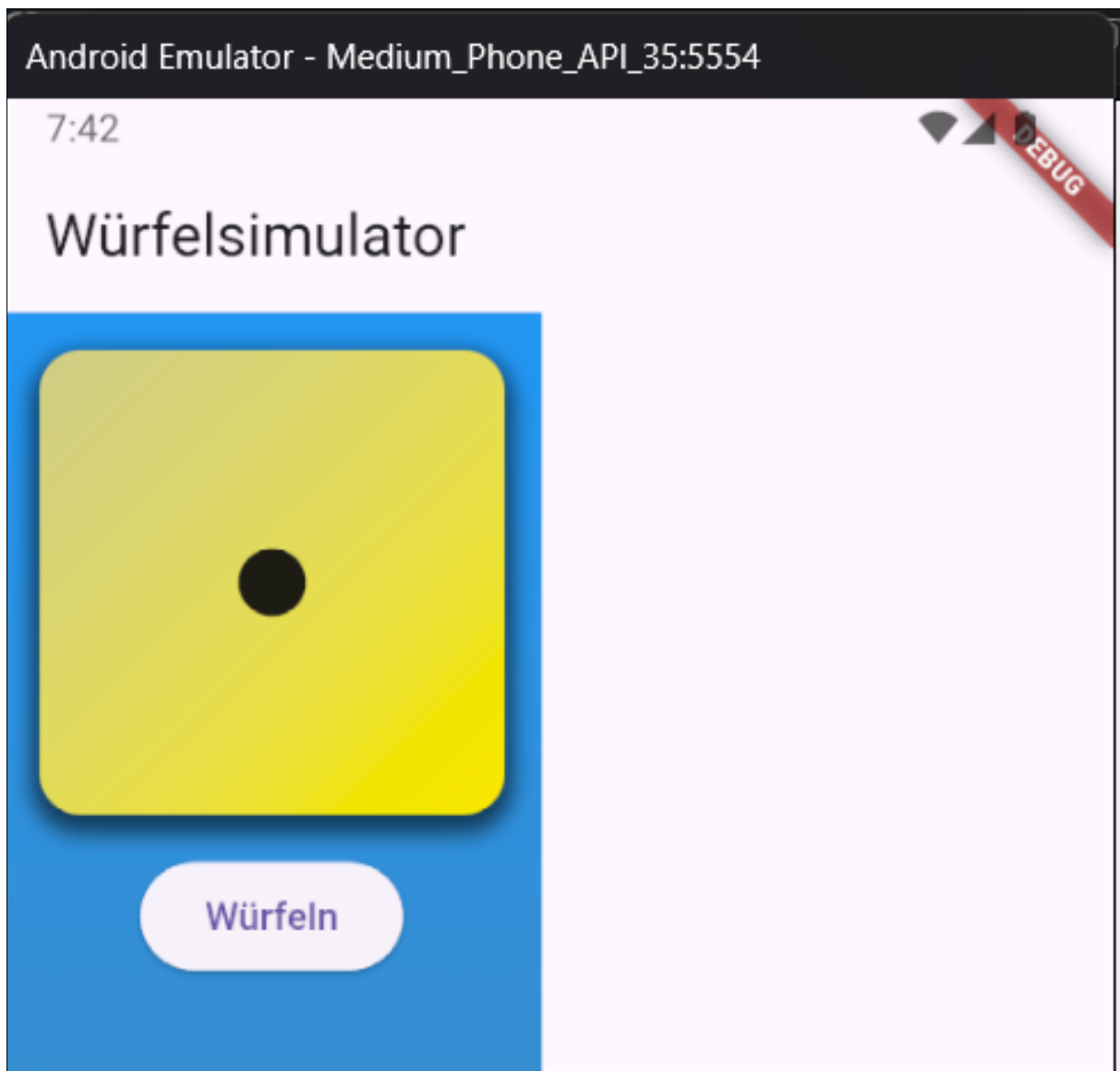
```
import 'package:flutter/material.dart';  
  
class DicePage extends StatelessWidget {  
  final String imagePath;  
  
  const DicePage({required this.imagePath, super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(  
      decoration: BoxDecoration(  
        gradient: LinearGradient(  
          begin: Alignment.topCenter,  
          end: Alignment.bottomCenter,  
          colors: [Colors.blue, Colors.blueGrey],  

```



```
    ),  
  ),  
  child: Column(  
    children: [  
      Image.asset(  
        imagePath,  
        width: 200,  
      ),  
      ElevatedButton(  
        onPressed: () { print('Button wurde geklickt'); },  
        child: Text('Würfel'),  
      ),  
    ],  
  ),  
);  
}  
}
```

Nun sieht unsere App in etwa so aus:



Das Column-Widget ist ein Layout-Widget, das seine Child-Widgets in einer vertikalen Sequenz anzeigt. Das **ElevatedButton**-Widget ist ein Material-Design-Button, der normalerweise in einer erhöhten Position angezeigt wird. Flutter stellt eine Vielzahl von Buttons zur Verfügung:

- **TextButton**: Ein Text-Button, der in einer flachen Position angezeigt wird.
- **ElevatedButton**: Ein erhöhter Button, der in einer erhöhten Position angezeigt wird.
- **OutlinedButton**: Ein umrandeter Button, der in einer flachen Position angezeigt wird.
- **IconButton**: Ein Button, der ein Icon anzeigt.

- **FloatingActionButton**: Ein kreisförmiger Button, der in der unteren rechten Ecke angezeigt wird.
- **DropDownButton**: Ein Dropdown-Button, der eine Liste von Elementen anzeigt.

Für den onPressed-Parameter des **ElevatedButton**-Widgets verwenden wir eine **anonyme Funktion**, die beim Klicken auf den Button ausgeführt wird. Alternativ können wir auch eine benannte Funktion verwenden:

```
import 'package:flutter/material.dart';

class DicePage extends StatelessWidget {
  final String imagePath;

  const DicePage({required this.imagePath, super.key});

  void rollDice() {
    print('Button wurde geklickt');
  }

  @override
  Widget build(BuildContext context) {
    return Container(
      decoration: BoxDecoration(
        gradient: LinearGradient(
          begin: Alignment.topCenter,
          end: Alignment.bottomCenter,
          colors: [Colors.blue, Colors.blueGrey],
        ),
      ),
      child: Column(
        children: [
```

```
        Image.asset(
          imagePath,
          width: 200,
        ),
        ElevatedButton(
          onPressed: rollDice,
          child: Text('Würfeln'),
        ),
      ],
    ),
  );
}
```

Styling

Nun kümmern wir uns darum, dass alles korrekt angeordnet und gestylt wird. Zentrieren wir dazu zuerst das Bild und den Button mittig:

```
...
child: Column(
  mainAxisAlignment: MainAxisAlignment.min,
  children: [
...

```

Nun ist der Inhalt zwar vertikal zentriert (da das `Column`-Widget standardmäßig die volle verfügbare Höhe verwendet), aber noch nicht horizontal. Dazu können wir das `Column`-Widget z.B. in ein `Center`-Widget einbetten:

```
...
child: Center(
```

```
child: Column(  
  mainAxisAlignment: MainAxisAlignment.min,  
  children: [  
    ...  
  ],  
),
```

Nun ist alles ordentlich vertikal und horizontal zentriert. Wir können nun noch den Button etwas stylen und den Abstand zum Würfel erhöhen:

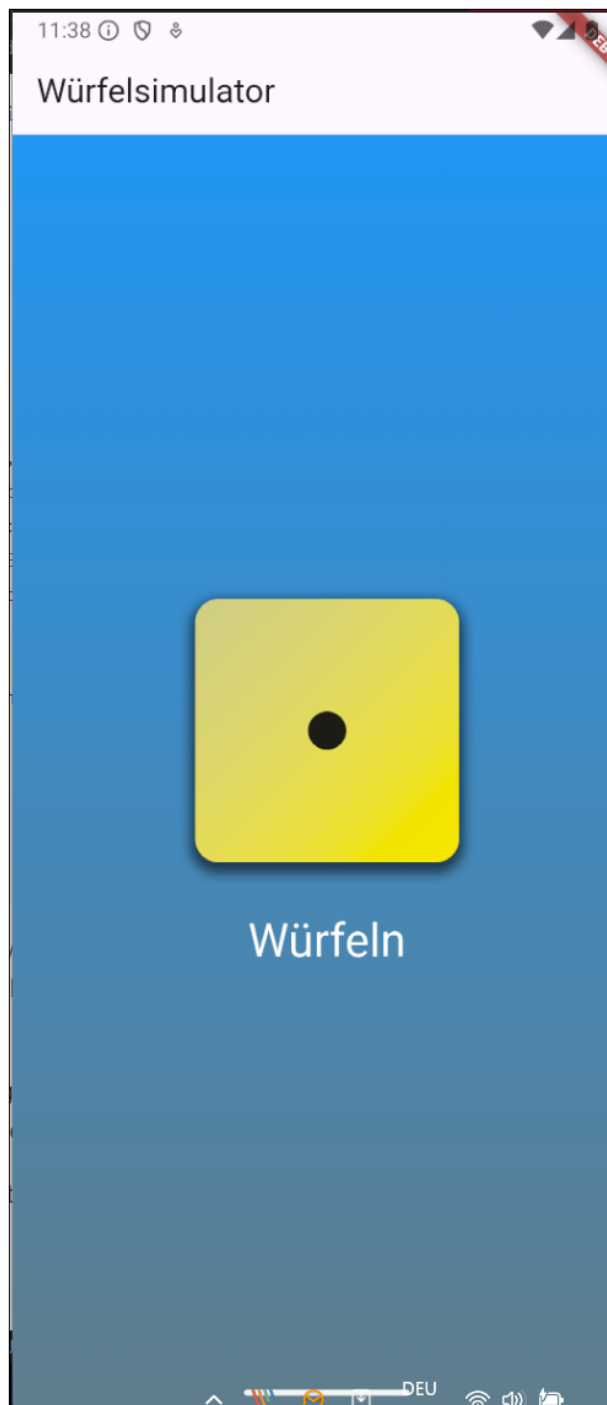
```
...  
child: Center(  
  child: Column(  
    mainAxisAlignment: MainAxisAlignment.min,  
    children: [  
      Image.asset(  
        imagePath,  
        width: 200,  
      ),  
      // SizedBox(  
      //   height: 20,  
      // ),  
      TextButton(  
        onPressed: rollDice,  
        style: TextButton.styleFrom(  
          foregroundColor: Colors.white,  
          padding: const EdgeInsets.only(top: 20),  
          textStyle: const TextStyle(  
            fontSize: 30,  
          )),  
        child: Text('Würfel'),  
      ),  
    ],  
  ),  
),
```

```
    ),  
  ));  
  ...
```

Das **SizedBox**-Widget ist ein Layout-Widget, das einen leeren Raum zwischen seinen Kindern erzeugt. Wir können die Höhe des **SizedBox**-Widgets mit dem **height**-Parameter ändern.

Der Unterschied zur Verwendung von **padding** ist, dass **padding** den Inhalt des Widgets verschiebt, während **SizedBox** den Raum zwischen den Widgets ändert.

Nun sieht unsere App schon deutlich besser aus:



Würfeln (noch falsch umgesetzt)

Nun wollen wir die Würfelfunktionalität implementieren. Das sollte für uns in der fünften Klasse kein Problem darstellen.

Starten wir also, indem wir uns in einer Variable merken, welches Bild gerade angezeigt werden soll:

```
...  
class DicePage extends StatelessWidget {  
  String imagePath;  
  
  DicePage({required this.imagePath, super.key});  
  ...  
}
```

Wir verwenden also die schon existierende Variable `imagePath`. Diese ist nun nicht mehr `const` - der Konstruktor auch nicht mehr.

Nun ändern wir den Pfad des Bildes, wenn der Button gedrückt wird:

```
...  
void rollDice() {  
  imagePath = 'assets/dice-images/dice-2.png';  
  print('Button wurde geklickt');  
}  
...
```

Wenn wir nun auf den Button klicken ... passiert gar nichts.

Das liegt daran, dass Flutter nicht weiß, dass sich der Inhalt des **DicePage**-Widgets geändert hat. Es hat sich also sein Status, sein **state** geändert. Wir müssen also das **StatelessWidget**-Widget in ein **StatefulWidget**-Widget umwandeln.

StatefulWidget

Ein **StatefulWidget** ist ein Widget, das seinen Zustand ändern kann. Es besteht aus zwei Klassen: einer **StatefulWidget**-Klasse und einer **State**-Klasse. Die **StatefulWidget**-Klasse ist für die Darstellung des Widgets, während die **State**-Klasse für die Verwaltung des Zustands des Widgets verantwortlich ist.

In unserem Beispiel ist der Großteil des ursprünglichen *state-less* Widgets unveränderlich, nur der Pfad des Bildes ändert sich. Daher können wir das **DicePage**-Widget in zwei kleinere Widgets aufteilen.

Erstellen wir dazu eine neue Datei `dice_roller.dart` mit folgendem Inhalt:

```
import 'package:flutter/material.dart';

class DiceRoller extends StatefulWidget {
  const DiceRoller({super.key});

  @override
  State<DiceRoller> createState() {
    return _DiceRollerState();
  }
}

class _DiceRollerState extends State<DiceRoller> {
  var imagePath = 'assets/dice-images/dice-1.png';

  @override
  Widget build(BuildContext context) {
    return Column(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
```

```
        Image.asset(
          imagePath,
          width: 200,
        ),
        TextButton(
          onPressed: rollDice,
          style: TextButton.styleFrom(
            foregroundColor: Colors.white,
            padding: const EdgeInsets.only(top: 20),
            textStyle: const TextStyle(
              fontSize: 30,
            ),
          ),
          child: Text('Würfeln'),
        ),
      ],
    );
  }

  void rollDice() {
    imagePath = 'assets/dice-images/dice-2.png';
  }
}
```

Wir verlagern also das gesamte Column-Widget in die **State**-Klasse. Der Pfad des Bildes wird in einem **State**-Objekt gespeichert und kann bei Bedarf geändert werden.

Nun müssen wir noch sicherstellen, dass unsere neue Klasse DiceRoller im DicePage-Widget verwendet wird:

```
import 'package:flutter/material.dart';
import 'package:wuerfelsimulator/dice_roller.dart';
```

```
class DicePage extends StatelessWidget {
  DicePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Container(
      decoration: BoxDecoration(
        gradient: LinearGradient(
          begin: Alignment.topCenter,
          end: Alignment.bottomCenter,
          colors: [Colors.blue, Colors.blueGrey],
        ),
      ),
      child: Center(
        child: DiceRoller(),
      ));
  }
}
```

Wir entfernen in diesem Zuge auch den Konstruktor `imagePath` aus dem `DicePage`-Widget. Dadurch muss auch noch die `main.dart`-Datei angepasst werden:

```
import 'package:flutter/material.dart';
import 'package:wuerfelsimulator/dice_page.dart';

void main() {
  runApp(MaterialApp(
    home: Scaffold(
      appBar: AppBar(
        title: Text('Würfelsimulator'),
      ),
    ),
```

```
        body: DicePage(),  
      ),  
    ));  
  }
```

Wenn wir nun die App aktualisieren und den Würfeln-Button drücken ... passiert immer noch nichts. Warum ist das so?

setState()-Methode

In Flutter wird der Zustand eines **StatefulWidget**-Widgets in einer separaten **State**-Klasse gespeichert. Wenn sich der Zustand des Widgets ändert, muss **explizit** die **setState()**-Methode aufgerufen werden, um Flutter mitzuteilen, dass sich der Zustand geändert hat und das Widget neu gerendert werden muss.

Ändern wir also die rollDice-Methode in der _DiceRollerState-Klasse:

```
void rollDice() {  
  setState(() {  
    imagePath = 'assets/dice-images/dice-2.png';  
  });  
}
```

Die **setState()**-Methode nimmt eine Funktion als Parameter, die den neuen Zustand des Widgets definiert. Wenn die **setState()**-Methode aufgerufen wird, ruft Flutter die **build**-Methode des Widgets erneut auf, um das Widget neu zu rendern.

Nun sollte beim Drücken des Buttons das Bild des Würfels geändert werden.



Zufällige Zahlen generieren

Wir wollen natürlich nicht immer das gleiche Bild anzeigen, sondern eine zufällige Zahl zwischen 1 und 6 generieren. Dazu können wir die **Random**-Klasse verwenden:

```
import 'dart:math';  
  
import 'package:flutter/material.dart';  
  
final _random = Random();  
  
class DiceRoller extends StatefulWidget {  
  const DiceRoller({super.key});
```

```
@override
State<DiceRoller> createState() {
  return _DiceRollerState();
}

class _DiceRollerState extends State<DiceRoller> {
  var currentDiceRoll = 1;

  void rollDice() {
    setState(() {
      currentDiceRoll = _random.nextInt(6) + 1;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        Image.asset(
          'assets/dice-images/dice- $\{currentDiceRoll\}$ .png',
          width: 200,
        ),
        TextButton(
          onPressed: rollDice,
          style: TextButton.styleFrom(
            foregroundColor: Colors.white,
            padding: const EdgeInsets.only(top: 20),
          ),
        ),
      ],
    );
  }
}
```

```
        textStyle: const TextStyle(
          fontSize: 30,
        )),
        child: Text('Würfeln'),
      ),
    ],
  );
}
```

Aufgabe

Erweitere den Würfelsimulator um ein weiteres Widget, welches unterhalb des Buttons angezeigt wird und die **Gesamtzahl** der bisher gewürfelten Augen anzeigt. Mithilfe eines weiteren Buttons soll der Benutzer die Gesamtzahl der Augen **zurücksetzen** können.

Mögliche Lösung

Um dieses Problem zu lösen, müssen wir eine Möglichkeit schaffen, den gemeinsamen Status beider Widgets, also die gewürfelte Augenzahl, an einer Stelle zu speichern, auf den beide Widgets zugreifen können. Man sagt dazu auch **"lifting the state up"**.

Wenn wir den nun gemeinsamen Status im Konstruktor beider Widgets (jenes zum Würfeln und jenes zum Mitzählen) übergeben, können beide Widgets auf den gemeinsamen Status zugreifen. Dem Würfeln-Widget müssen wir aber auch noch eine Funktion übergeben, die den Status ändern kann.

Erste Lösungs-Idee:

```
class _DiceRollerState extends State<DiceRoller> {
```

```
int _currentDiceRoll = 1;

void _rollDice() {
  setState(() {
    _currentDiceRoll = _random.nextInt(6) + 1;
  });
  print('Button "Würfel" gedrückt');
}

@override
Widget build(BuildContext context) {
  return Center(
    child: Column(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        HandleDice(currentDiceRoll: _currentDiceRoll, rollDice:
→ _rollDice),
        SizedBox(height: 20),
        CountRolls(currentDiceRoll: _currentDiceRoll),
      ],
    ),
  );
}
```

Dem `HandleDice`-Widget übergeben wir also die aktuelle Augenzahl und die Funktion zum Würfeln. Hier genügt nun ein `StatelessWidget`:

```
class HandleDice extends StatelessWidget {
  final int currentDiceRoll;
  final void Function() rollDice;
```



```
const HandleDice(  
  {required this.currentDiceRoll, required this.rollDice,  
   ↪ super.key});  
  
@override  
Widget build(BuildContext context) {  
  return Column(children: [  
    Image.asset(  
      'assets/dice-images/dice-$currentDiceRoll.png',  
      width: 200,  
    ),  
    SizedBox(height: 20),  
    TextButton(  
      onPressed: rollDice,  
      style: TextButton.styleFrom(  
        foregroundColor: Colors.white,  
        textStyle: TextStyle(fontSize: 30, fontWeight:  
↪ FontWeight.bold),  
      ),  
      child: Text('Würfeln'))  
  ]);  
}
```

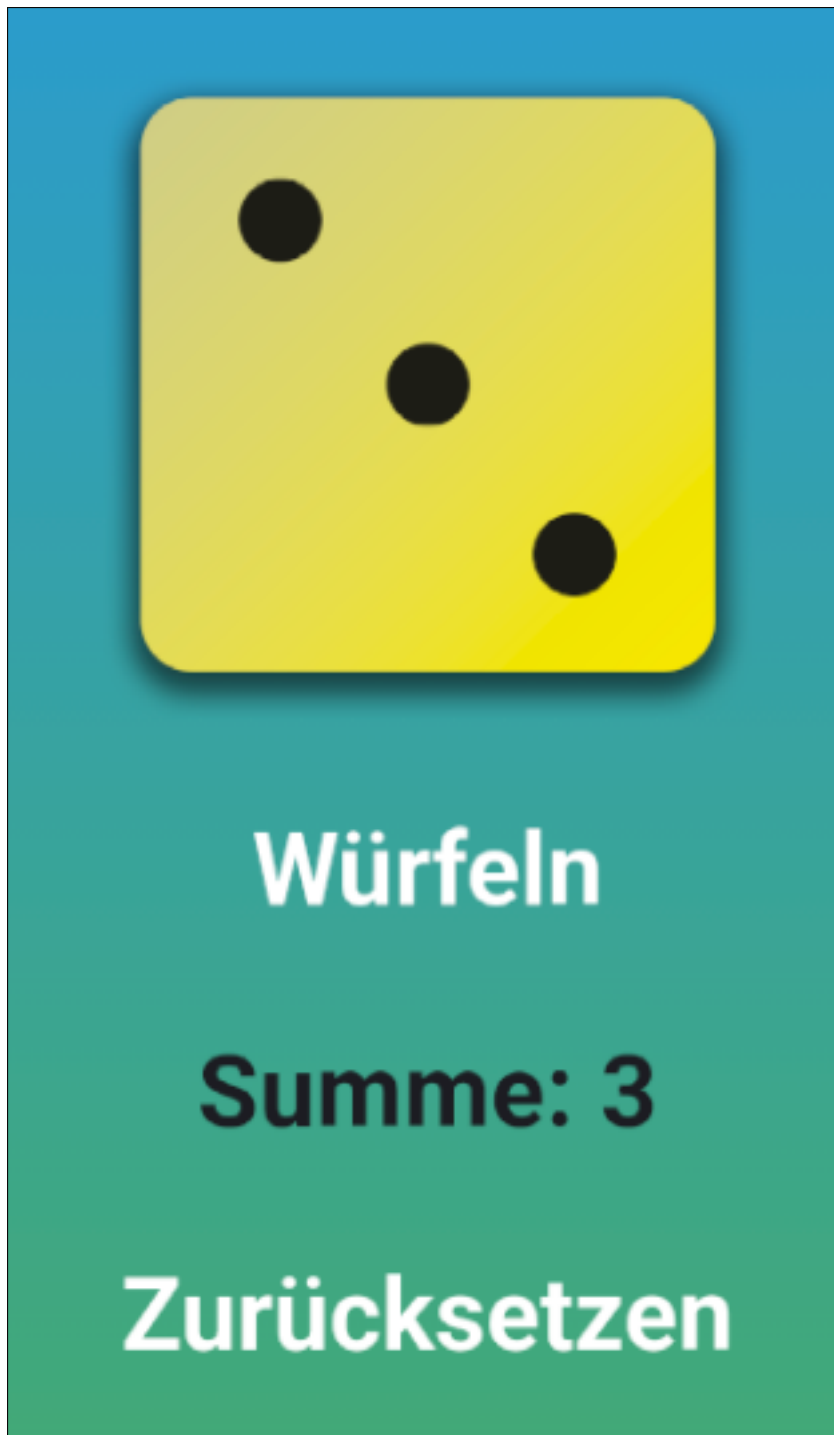
Das CountRolls-Widget soll dann die Gesamtzahl der Augen anzeigen und einen Button zum Zurücksetzen bereitstellen. Beginnen wir mit einem StatelessWidget, welches nur die Anzeige der zuletzt gewürfelten Augenzahl und den Buttons zum Zurücksetzen enthält:

```
class CountRolls extends StatelessWidget {  
  final int currentDiceRoll;
```

```
const CountRolls({required this.currentDiceRoll, super.key});

@override
Widget build(BuildContext context) {
  return Column(children: [
    Text('Summe: $currentDiceRoll',
      style: TextStyle(fontSize: 30, fontWeight:
↪ FontWeight.bold)),
    SizedBox(height: 20),
    TextButton(
      onPressed: () {},
      style: TextButton.styleFrom(
        foregroundColor: Colors.white,
        textStyle: TextStyle(fontSize: 30, fontWeight:
↪ FontWeight.bold),
      ),
      child: Text('Zurücksetzen'))
  ]);
}
```

Nun sieht unser Würfelsimulator so aus:



Wenn wir nun aber mitzählen möchten, müssen wir den Status in ein **StatefulWidget**-Widget heben. Wir können also das `CountRolls`-Widget in ein `StatefulWidget`-Widget umwandeln:

```
class CountRolls extends StatefulWidget {  
  final int currentDiceRoll;  
  
  const CountRolls({required this.currentDiceRoll, super.key});  
  
  @override  
  State<CountRolls> createState() {  
    return _CountRollsState();  
  }  
}
```

```
class _CountRollsState extends State<CountRolls> {  
  int _totalRolls = 0;  
  
  void _resetRolls() {  
    setState(() {  
      _totalRolls = 0;  
    });  
  }  
}
```

```
@override  
Widget build(BuildContext context) {  
  return Column(children: [  
    Text('Summe: ${widget.currentDiceRoll}',  
      style: TextStyle(fontSize: 30, fontWeight:  
→ FontWeight.bold)),  
    SizedBox(height: 20),  
    TextButton(  
      onPressed: _resetRolls,  
      style: TextButton.styleFrom(  

```

```
        foregroundColor: Colors.white,  
        textStyle: TextStyle(fontSize: 30, fontWeight:  
→   FontWeight.bold),  
      ),  
      child: Text('Zurücksetzen'))  
    ]));  
  }  
}
```

Wie erhöhen wir nun aber die Gesamtzahl der Augen? Auslöser ist ja `_rollDice()`-Methode im `HandleDice`-Widget. Diese weiß aber nichts von der Gesamtzahl der Augen. Wir müssen also die Gesamtzahl der Augen in das `HandleDice`-Widget heben.

```
class _DiceRollerState extends State<DiceRoller> {  
  int _currentDiceRoll = 1;  
  int _totalRolls = 0;  
  
  void _rollDice() {  
    setState(() {  
      _currentDiceRoll = _random.nextInt(6) + 1;  
      _totalRolls += _currentDiceRoll;  
    });  
    print('Button "Würfeln" gedrückt');  
  }  
}
```

@override

```
Widget build(BuildContext context) {  
  return Center(  
    child: Column(  
      mainAxisAlignment: MainAxisAlignment.min,  
      children: [  

```

```
        HandleDice(
          currentDiceRoll: _currentDiceRoll,
          rollDice: _rollDice),
        SizedBox(height: 20),
        CountRolls(totalRolls: _totalRolls),
      ],
    ),
  );
}
```

Dem CountRolls-Widget übergeben wir also die Gesamtzahl der Augen. Wir speichern darin also keinen Status mehr, sprich wieder können daraus wieder ein StatelessWidget machen:

```
class CountRolls extends StatelessWidget {
  final int totalRolls;

  const CountRolls({required this.totalRolls, super.key});

  @override
  Widget build(BuildContext context) {
    return Column(children: [
      Text('Summe: $totalRolls',
        style: TextStyle(fontSize: 30, fontWeight:
→ FontWeight.bold)),
      SizedBox(height: 20),
      TextButton(
        onPressed: () {},
        style: TextButton.styleFrom(
          foregroundColor: Colors.white,
```

```
        textStyle: TextStyle(fontSize: 30, fontWeight:
→ FontWeight.bold),
      ),
      child: Text('Zurücksetzen'))
    ]);
  }
}
```

Was noch fehlt ist das Zurücksetzen der Gesamtzahl der Augen. Die Gesamtzahl wird nun aber wieder im DiceRoller-Widget erhöht. Wir müssen also die Funktion zum Zurücksetzen in das CountRolls-Widget heben:

```
class CountRolls extends StatelessWidget {
  final int totalRolls;
  final void Function() resetRolls;

  const CountRolls({required this.totalRolls, required
→ this.resetRolls, super.key});

  @override
  Widget build(BuildContext context) {
    return Column(children: [
      Text('Summe: $totalRolls',
        style: TextStyle(fontSize: 30, fontWeight:
→ FontWeight.bold)),
      SizedBox(height: 20),
      TextButton(
        onPressed: resetRolls,
        style: TextButton.styleFrom(
          foregroundColor: Colors.white,
```

```
        textStyle: TextStyle(fontSize: 30, fontWeight:
↪  FontWeight.bold),
      ),
      child: Text('Zurücksetzen'))
    ]);
  }
}
```

Das DiceRoller-Widget sieht nun so aus:

```
class DiceRoller extends StatefulWidget {
  const DiceRoller({super.key});

  @override
  State<DiceRoller> createState() {
    return _DiceRollerState();
  }
}

class _DiceRollerState extends State<DiceRoller> {
  int _currentDiceRoll = 1;
  int _totalRolls = 0;

  void _rollDice() {
    setState() {
      _currentDiceRoll = _random.nextInt(6) + 1;
      _totalRolls += _currentDiceRoll;
    });
    print('Button "Würfel" gedrückt');
  }
}
```



```
void _resetRolls() {  
  setState(() {  
    _totalRolls = 0;  
  });  
}
```

@override

```
Widget build(BuildContext context) {  
  return Center(  
    child: Column(  
      mainAxisAlignment: MainAxisAlignment.min,  
      children: [  
        HandleDice(  
          currentDiceRoll: _currentDiceRoll,  
          rollDice: _rollDice),  
        SizedBox(height: 20),  
        CountRolls(totalRolls: _totalRolls, resetRolls:  
→ _resetRolls),  
      ],  
    ),  
  );  
}
```

Nun funktioniert der Würfelsimulator wie gewünscht...jedoch so ganz optimal ist diese Lösung nach wie vor nicht.

Lösung mithilfe eines Provider

Eine bessere Lösung wäre die Verwendung eines **Providers**. Ein **Provider** ist ein Widget, das Daten an seine Kind-Widgets bereitstellt. Ein **Provider** kann auch den Zustand des Widgets verwalten und die Kind-Widgets benachrichtigen, wenn sich der Zustand ändert.

Es setzt dabei das uns schon bekannte **Observer-Pattern** um. Ein **Provider** ist also ein **Subject** und die Widgets, die auf die Daten zugreifen, sind die **Observers**.

Betrachten wir nochmal unsere ursprüngliche Idee der Aufteilung in zwei Widgets:

- Das erste Widget soll sich nur um das Würfeln und die Anzeige der zugehörigen Grafik kümmern.
- Das zweite Widget soll sich um die Anzeige der Gesamtzahl der Augen und das Zurücksetzen dieser kümmern.

Das zweite Widget soll also **informiert** werden, wenn eine neue Augenzahl im ersten Widget gewürfelt wurde.

Provider-Paket installieren

Um das **Provider**-Package zu verwenden, fügen wir es zu unserer `pubspec.yaml`-Datei hinzu. Am einfachsten geht das über folgenden Befehl im Terminal:

```
flutter pub add provider
```

Nun können wir folgendes Paket importieren und verwenden:

```
import 'package:provider/provider.dart';
```

ChangeNotifier-Klasse erstellen

Erstellen wir eine neue Klasse `DiceRollerModel`. Diese speichert den zuletzt gewürfelten Wert und stellt eine Methode zum Würfeln zur Verfügung. Wurde ein neuer Wert gewürfelt,

benachrichtigt die Klasse alle Observer.

```
import 'dart:math';
import 'package:flutter/material.dart';

final _random = Random();

class DiceRollerModel extends ChangeNotifier {
  int _currentDiceRoll = 1;

  int get currentDiceRoll => _currentDiceRoll;

  void rollDice() {
    _currentDiceRoll = _random.nextInt(6) + 1;
    notifyListeners();
  }
}
```

ChangeNotifierProvider einbauen

Wir müssen nun den ChangeNotifierProvider in unserer App einbauen. Dieser wird in der main.dart-Datei eingebaut:

```
import 'package:dice_app/dice_roller_model.dart';
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:dice_app/dice_page.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
```

```
    create: (context) => DiceRollerModel(),
    child: MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: Text('Würfelsimulator'),
        ),
        body: DicePage(),
      ),
    ),
  );
}
```

Consumer-Widget verwenden

Nun können wir in den Widgets, die auf die Daten zugreifen sollen, das Consumer-Widget verwenden. Das Consumer-Widget hört auf Änderungen des **Providers** und ruft die `builder`-Funktion auf, wenn sich der Zustand des **Providers** ändert.

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:dice_app/dice_roller_model.dart';

class HandleDice extends StatelessWidget {

  const HandleDice({super.key});

  @override
  Widget build(BuildContext context) {
    return Consumer<DiceRollerModel>(
      builder: (context, diceRollerModel, child) {
```

```
    return Column(children: [
      Image.asset(
        ↪ 'assets/dice-images/dice- $\{diceRollerModel.currentDiceRoll\}$ .png'
        width: 200,
      ),
      SizedBox(height: 20),
      TextButton(
        onPressed: () {
          diceRollerModel.rollDice();
        },
        style: TextButton.styleFrom(
          foregroundColor: Colors.white,
          textStyle: TextStyle(fontSize: 30, fontWeight:
↪ FontWeight.bold),
        ),
        child: Text('Würfeln'))
    ]);
  },
);
}
```

Im CountRolls-Widget speichern wir nun die Gesamtzahl der Augen und reagieren darauf, wenn ein neuer Würfelwert gewürfelt wurde. Wir benötigen als ein StatefulWidget-Widget. Wenn sich der Wert ändert, erhöhen wir die Gesamtzahl der Augen.

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:dice_app/dice_roller_model.dart';
```

```
class CountRolls extends StatefulWidget {
  const CountRolls({super.key});

  @override
  State<CountRolls> createState() {
    return _CountRollsState();
  }
}

class _CountRollsState extends State<CountRolls> {
  int _totalRolls = 0;

  @override
  Widget build(BuildContext context) {
    return Consumer<DiceRollerModel>(
      builder: (context, diceRollerModel, child) {
        _totalRolls += diceRollerModel.currentDiceRoll;
        return Column(children: [
          Text('Summe: $_totalRolls',
            style: TextStyle(fontSize: 30, fontWeight:
→ FontWeight.bold)),
          SizedBox(height: 20),
          TextButton(
            onPressed: () {
              setState(() {
                _totalRolls = -diceRollerModel.currentDiceRoll;
              });
            },
            style: TextButton.styleFrom(
              foregroundColor: Colors.white,
```

```
                textStyle: TextStyle(fontSize: 30, fontWeight:
↪   FontWeight.bold),
                ),
                child: Text('Zurücksetzen'))
            ]);
        },
    );
}
```

Das DicePage-Widget sieht nun so aus:

```
import 'package:flutter/material.dart';
import 'package:dice_app/handle_dice.dart';
import 'package:dice_app/count_rolls.dart';

class DicePage extends StatelessWidget {
  DicePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Container(
      decoration: BoxDecoration(
        gradient: LinearGradient(
          begin: Alignment.topCenter,
          end: Alignment.bottomCenter,
          colors: [Colors.blue, Colors.green],
        ),
      ),
      child: Center(
        child: Column(
```

```
        mainAxisAlignment: MainAxisAlignment.min,  
        children: [  
          HandleDice(),  
          SizedBox(height: 20),  
          CountRolls(),  
        ],  
      ),  
    ),  
  );  
}  
}
```

Dadurch haben wir eine möglichst **lose Kopplung** zwischen den beiden Widgets erzielt. Das `HandleDice`-Widget kümmert sich nur um das Würfeln und die Anzeige des Würfelergebnisses. Das `CountRolls`-Widget kümmert sich nur um die Anzeige der Gesamtzahl der Augen und das Zurücksetzen.